Tugas Kelompok ke-3

Week 8

Data Structures and Algorithm Analysis



Amelia Rizqi Fadila - 2902824546

Rismanto - 2902811940

Primaaditya Redyananda – 2902814450

Putri Puspita Kartini Rahman - 2902820314

BINUS University Semester Ganjil

2026/2027 Periode 1

BINUS UNIVERSITY ONLINE

People
Innovation
Excellence

Tugas Kelompok ke-3

Week 8

Algorithmic Analysis, Mathematical Induction, and Recursive Functions

Learning Outcome:

- LO2: Analyze algorithm complexity: Students will be able to assess the time and space complexity of algorithms

---

Instruksi Pengerjaan

1. Buat program menggunakan Java dengan nama file PlaylistRekursif.java.
2. Gunakan array objek Lagu (dari tugas sebelumnya) untuk menyimpan data playlist.
3. Implementasikan fungsi-fungsi berbasis rekursif untuk melakukan operasi analisis pada playlist.
4. Tulis komentar di setiap fungsi utama untuk menjelaskan:
   - Tujuan fungsi
   - Base case dan recursive case
   - Kompleksitas waktunya
5. Tambahkan fitur pengukuran waktu eksekusi (time measurement) yang ada dalam Java.
6. Sertakan identitas kelompok di bagian atas file (nama, NIM, kelas).
7. Kumpulkan hasil dalam bentuk:

   - File `.java`
   - Screenshot hasil eksekusi setiap operasi
   - Link GitHub/Replit/video demo
   - Laporan PDF singkat yang berisi analisis *time and growth rate* untuk setiap fungsi rekursif.
   - Pastikan semua berkas diunggah ke LMS.

People
Innovation
Excellence

Deskripsi Tugas

Kelompok diminta untuk mengembangkan sistem manajemen playlist musik yang telah dibuat pada tugas sebelumnya, dengan menambahkan kemampuan analisis rekursif dan pengukuran efisiensi algoritmik.

Program ini harus mampu:

1) Menghitung total durasi playlist menggunakan rekursi.
2) Menampilkan daftar lagu secara terbalik (reverse traversal) menggunakan rekursi.
3) Menentukan lagu dengan durasi terpanjang menggunakan rekursi.
4) Menganalisis kompleksitas waktu dan growth rate dari fungsi-fungsi tersebut.

Struktur Program yang Diharapkan

1. **Class Lagu**
   - Gunakan kembali dari Tugas Kelompok 1 (atribut: judul, artis, durasi).
2. **Class PlaylistRekursif**
   - Memiliki atribut Lagu[] playlist.
   - Memiliki fungsi-fungsi:
     - ✓ static double totalDurasi(Lagu[] list, int n);
     - ✓ static void tampilkanMundur(Lagu[] list, int index);
     - ✓ static double cariDurasiTerpanjang(Lagu[] list, int index);
   - Tambahkan pengukuran waktu eksekusi untuk masing-masing fungsi.

BINUS UNIVERSITY ONLINE

People Innovation Excellence

Soal:

1. Implementasi Fungsi Rekursif Dasar – (Bobot 40%)
   - `totalDurasi()` → menghitung total seluruh durasi lagu.
   - `tampilkanMundur()` → menampilkan lagu dari akhir ke awal.
   - `cariDurasiTerpanjang()` → mencari lagu dengan durasi terpanjang.

```
Total Durasi Lagu = 21.36 menit
Daftar Lagu (Terbalik):
Fix You - Coldplay (4.52)
Shivers - Ed Sheeran (3.50)
Perfect - Ed Sheeran (4.23)
Lagu Terpanjang: Fix You (4.52 menit)
```

2. Analisis Kompleksitas Waktu (Time and Growth Rate) – (Bobot 30%)

   Tuliskan analisis **Big-O** untuk setiap fungsi:

Fungsi	Base Case	Growth Rate	Kompleksitas
totalDurasi()			
tampilkanMundur()			
cariDurasiTerpanjang()			

   Sertakan grafik sederhana (dalam laporan) yang menggambarkan *growth rate* linear terhadap ukuran data playlist (misal, n = 2, 4, 6, 8, 10).

3. Eksperimen Pengukuran Waktu Eksekusi (Bobot 20%)
   - Ukur dan tampilkan **waktu eksekusi (dalam milidetik)** untuk setiap fungsi.
   - Jalankan program dengan jumlah data berbeda (misal: 3, 5, dan 10 lagu).
   - Bandingkan hasil waktu eksekusi dan buat tabel analisis di laporan:

Jumlah Lagu (n)	totalDurasi()	tampilkanMundur()	cariDurasiTerpanjang()
3	1 ms	1 ms	0 ms
5	2 ms	2 ms	1 ms
10	4 ms	4 ms	3 ms

4. Kerapian Program dan Dokumentasi (Bobot 10%)

   a. Kode tersusun rapi, menggunakan indentasi yang konsisten.

   b. Setiap fungsi utama memiliki komentar.

   c. Output program tampil jelas dan mudah dibaca.

Output yang diharapkan

```
=== ANALISIS REKURSIF PLAYLIST ===
Jumlah lagu : 5
Total durasi : 20.35 menit
Lagu terpanjang : "Viva La Vida" - Coldplay (4.52 menit)

Daftar lagu (ditampilkan terbalik):
5. Viva La Vida
4. Yellow
3. Fix You
2. Shivers
1. Perfect

Execution Time (totalDurasi): 3 ms
Execution Time (tampilkanMundur): 2 ms
Execution Time (cariDurasiTerpanjang): 3 ms
```

1. Implementasi Fungsi Rekursif Dasar

Lagu.java

```
package playlist;

public class Lagu {
    private String judul;
    private String artis;
    private double durasi;

    public Lagu(String judul, String artis, double durasi) {
        this.judul = judul;
        this.artis = artis;
        this.durasi = durasi;
    }

    public String getJudul() {
        return judul;
    }

    public void setJudul(String judul) {
        this.judul = judul;
    }

    public String getArtis() {
        return artis;
    }

    public void setArtis(String artis) {
        this.artis = artis;
    }

    public double getDurasi() {
        return durasi;
    }

    public void setDurasi(double durasi) {
        this.durasi = durasi;
    }

    public void tampilkanInfo() {
        System.out.println("Judul : " + judul);
        System.out.println("Artis : " + artis);
        System.out.printf("Durasi : %.2f menit%n", durasi);
    }
```

```
}
```

PlaylistRekursif.java

```
package playlist;
public class PlaylistRekursif {
    static Lagu[] playlist;
    /*
     * Menghitung total durasi seluruh lagu dalam playlist
     * menggunakan rekursi.
     *
     * Base case:
     * Jika n == 0, tidak ada lagu yang dihitung,
     * fungsi mengembalikan 0.
     *
     * Recursive case:
     * Ambil durasi lagu ke-n (list[n - 1]), lalu tambahkan
     * dengan totalDurasi(list, n - 1) untuk lagu-lagu sebelumnya.
     *
     * Kompleksitas waktu: O(n)
     * Kompleksitas ruang: O(n)
     */

    static double totalDurasi(Lagu[] list, int n) {
        if (n == 0) {
            return 0;
        }
        return list[n - 1].getDurasi() + totalDurasi(list, n - 1);
    }

    /*
     * Menampilkan daftar lagu secara terbalik (reverse traversal)
     * menggunakan rekursi, dari lagu terakhir sampai lagu pertama.
     *
     * Base case:
     * Jika index < 0, tidak ada lagu lagi yang ditampilkan,
     * fungsi berhenti (return).
```

```
     *
     * Recursive case:
     * Tampilkan lagu pada posisi index, lalu panggil
     * tampilkanMundur(list, index - 1) untuk lagu sebelumnya.
     *
     * Kompleksitas waktu: O(n)
     * Kompleksitas ruang: O(n)
     */
    static void tampilkanMundur(Lagu[] list, int index) {
        if (index < 0) {
            return;
        }
        System.out.println((index + 1) + ". " + list[index].getJudul());
        tampilkanMundur(list, index - 1);
    }

    /*
     * Mencari durasi lagu terpanjang menggunakan rekursi.
     *
     * Base case:
     * Jika index == 0, kembalikan durasi lagu pertama.
     *
     * Recursive case:
     * Cari durasi terpanjang dari elemen sebelumnya,
     * kemudian bandingkan dengan durasi lagu saat ini.
     *
     * Kompleksitas waktu: O(n)
     * Kompleksitas ruang: O(n)
     */
    static double cariDurasiTerpanjang(Lagu[] list, int index) {
        if (index == 0) {
            return list[0].getDurasi();
        }

        double durasiSebelumnya =
                cariDurasiTerpanjang(list, index - 1);

        if (list[index].getDurasi() > durasiSebelumnya) {
            return list[index].getDurasi();
```

```
        }

        return durasiSebelumnya;
    }

    /*
     * Mencari objek Lagu dengan durasi terpanjang.
     * Fungsi ini menggunakan hasil cariDurasiTerpanjang()
     * untuk menentukan durasi maksimum, lalu mencari objeknya.
     *
     * Kompleksitas waktu: O(n)
     */
    static Lagu cariLaguTerpanjang(Lagu[] list, int jumlahLagu) {
        if (jumlahLagu == 0) {
            return null;
        }

        double durasiMaksimum =
                cariDurasiTerpanjang(list, jumlahLagu - 1);

        for (int i = 0; i < jumlahLagu; i++) {
            if (list[i].getDurasi() == durasiMaksimum) {
                return list[i];
            }
        }

        return null;
    }

    public static void main(String[] args) {
        playlist = new Lagu[5];
        playlist[0] = new Lagu("Perfect", "Ed Sheeran", 3.50);
        playlist[1] = new Lagu("Shivers", "Ed Sheeran", 3.80);
        playlist[2] = new Lagu("Fix You", "Coldplay", 4.15);
        playlist[3] = new Lagu("Yellow", "Coldplay", 4.38);
        playlist[4] = new Lagu("Viva La Vida", "Coldplay", 4.52);

        int jumlahLagu = playlist.length;
```

```
        System.out.println("=== ANALISIS REKURSIF PLAYLIST ===");
        System.out.println("Jumlah lagu : " + jumlahLagu);

        long s1 = System.nanoTime();
        double total = totalDurasi(playlist, jumlahLagu);
        long e1 = System.nanoTime();
        long t1 = (e1 - s1) / 1_000_000;
        System.out.println("Total durasi : " + String.format("%.2f", total)
+ " menit");

        long s2 = System.nanoTime();
        Lagu laguTerpanjang = cariLaguTerpanjang(playlist, playlist.length);
        long e2 = System.nanoTime();
        long t2 = (e2 - s2) / 1_000_000;

        if (laguTerpanjang != null) {
            System.out.println("Lagu terpanjang: " +
laguTerpanjang.getJudul());
            System.out.println("Artis: " + laguTerpanjang.getArtis());
            System.out.printf("Durasi: %.2f menit%n",
laguTerpanjang.getDurasi());
        }

        System.out.println();
        System.out.println("Daftar lagu (ditampilkan terbalik):");
        long s3 = System.nanoTime();
        tampilkanMundur(playlist, jumlahLagu - 1);
        long e3 = System.nanoTime();
        long t3 = (e3 - s3) / 1_000_000;
        System.out.println();

        System.out.println("Execution Time (totalDurasi): " + t1 + " ms");
        System.out.println("Execution Time (tampilkanMundur): " + t3 + "
ms");
        System.out.println("Execution Time (cariDurasiTerpanjang): " + t2 +
" ms");
    }
}
```

2. Analisis Kompleksitas Waktu (Time and Growth Rate)

- **Analisis totalDurasi()**

  Fungsi static double totalDurasi(Lagu[] list, int n) menjumlahkan durasi seluruh lagu dalam playlist secara rekursif. Parameter n menyatakan jumlah lagu yang dihitung, dimulai dari lagu pertama. Fungsi dipanggil pertama kali dengan n sama dengan jumlah lagu: totalDurasi(playlist, jumlahLagu).

  Base case dan recursive case:

  - Base case: n == 0. Tidak ada lagu yang tersisa untuk dihitung, sehingga fungsi mengembalikan 0 tanpa memanggil dirinya lagi.
  - Recursive case: n > 0. Fungsi mengambil durasi lagu ke-n (list[n - 1]), lalu menambahkannya dengan hasil totalDurasi(list, n - 1). Karena n selalu berkurang 1, base case pasti tercapai.

  Jejak pemanggilan untuk 3 lagu (Perfect 3.50, Shivers 3.80, Fix You 4.15):

Pemanggilan ke-	n	Kasus	Proses	Nilai yang dikembalikan
1	3	Recursive case	4.15 + totalDurasi(list, 2)	4.15 + 7.30 = 11.45
2	2	Recursive case	3.80 + totalDurasi(list, 1)	3.80 + 3.50 = 7.30
3	1	Recursive case	3.50 + totalDurasi(list, 0)	3.50 + 0 = 3.50
4	0	Base case	(berhenti)	0

  Penjumlahan baru dikerjakan setelah base case tercapai. Pemanggilan turun dari n = 3 sampai n = 0, lalu hasilnya dijumlahkan satu per satu saat kembali naik, sehingga total durasi 3 lagu adalah 11.45 menit.

Analisis kompleksitas waktu (Big-O):

Misalkan n adalah jumlah lagu yang dihitung. Setiap pemanggilan melakukan pekerjaan konstan c (satu pengecekan n, satu akses array, dan satu penjumlahan), lalu memanggil dirinya sekali dengan n berkurang 1. Base case hanya melakukan satu pengecekan dan satu return (konstan c_0). Persamaan rekurensnya:

$T(n) = T(n - 1) + c$, dengan $T(0) = c_0$

Penyelesaian dengan substitusi berulang:

$T(n) = T(n - 1) + c = T(n - 2) + 2c = T(n - 3) + 3c = \cdots = T(0) + n \cdot c = c \cdot n + c_0$

Pembuktian dengan induksi matematika bahwa jumlah pemanggilan $P(n) = n + 1$ untuk semua $n \geq 0$:

- Basis (n = 0): fungsi langsung masuk base case, sehingga P(0) = 1 = 0 + 1. Benar.
- Hipotesis: anggap $P(k) = k + 1$ benar untuk suatu $k \geq 0$.
- Langkah induksi: totalDurasi(list, k + 1) dihitung satu kali, ditambah P(k) pemanggilan dari totalDurasi(list, k), sehingga P(k + 1) = 1 + (k + 1) = (k + 1) + 1. Benar.

Kompleksitas ruang: setiap pemanggilan harus menunggu hasil pemanggilan berikutnya, sehingga call stack menumpuk hingga n + 1 frame. Kompleksitas ruangnya O(n). Pada playlist yang sangat besar (misalnya 100.000 lagu) hal ini dapat menyebabkan StackOverflowError, tetapi untuk ukuran playlist pada tugas ini tidak menjadi masalah.

Hasil Analisis Big-O: untuk n lagu terjadi n + 1 pemanggilan dengan pekerjaan konstan di setiap pemanggilan, sehingga waktu eksekusi bertambah sebanding dengan n. Growth rate-nya linear dan kompleksitas waktunya O(n).

Pengujian hasil perhitungan total durasi:

Hasil fungsi dibandingkan dengan perhitungan manual menggunakan data playlist pada program. Untuk pengujian 10 lagu ditambahkan 5 lagu uji: Photograph (4.18), Bad Habits (3.51), The Scientist (5.09), Thinking Out Loud (4.41), dan Paradise (4.38).

Jumlah lagu	Perhitungan manual	Output program	Jumlah pemanggilan	Status
0	0	0.00 menit	1	Sesuai
1	3.50	3.50 menit	2	Sesuai
3	3.50 + 3.80 + 4.15 = 11.45	11.45 menit	4	Sesuai
5	11.45 + 4.38 + 4.52 = 20.35	20.35 menit	6	Sesuai
10	20.35 + 4.18 + 3.51 + 5.09 + 4.41 + 4.38 = 41.92	41.92 menit	11	Sesuai

Semua hasil sesuai dengan perhitungan manual, termasuk kasus playlist kosong. Hasil untuk 5 lagu (20.35 menit) juga sama dengan output yang diharapkan pada soal. Jumlah pemanggilan di setiap pengujian selalu n + 1, sesuai dengan hasil analisis Big-O.

- **Analisis tampilkanMundur()**

Fungsi static void tampilkanMundur(Lagu[] list, int index) menampilkan daftar lagu dari akhir ke awal (reverse traversal). Fungsi dipanggil pertama kali dengan index lagu terakhir: tampilkanMundur(playlist, jumlahLagu - 1).

Base case dan recursive case:

- Base case: index < 0. Semua lagu sudah dicetak, sehingga fungsi berhenti tanpa memanggil dirinya lagi.
- Recursive case: index >= 0. Fungsi mencetak lagu ke-(index + 1), lalu memanggil tampilkanMundur(list, index - 1). Karena index selalu berkurang 1, base case pasti tercapai.

Jejak pemanggilan untuk 3 lagu (Perfect, Shivers, Fix You):

Pemanggilan ke-	Index	Kasus	Yang Dicetak
1	2	Recursive case	3. Fix You
2	1	Recursive case	2. Shivers
3	0	Recursive case	1. Perfect
4	-1	Base case	(berhenti)

Analisis kompleksitas waktu (Big-O):

Misalkan n adalah jumlah lagu. Setiap pemanggilan melakukan pekerjaan konstan c (satu pengecekan index dan satu println), lalu memanggil dirinya sekali dengan index berkurang 1. Base case hanya melakukan satu pengecekan (konstan c_0). Persamaan rekurennya:

$T(n) = T(n-1) + c$, dengan $T(0) = c_0$

Penyelesaian dengan substitusi berulang:

$T(n) = T(n-1) + c = T(n-2) + 2c = \cdots = T(0) + n \cdot c = c \cdot n + c_0$

Pembuktian dengan induksi matematika bahwa $T(n) = c \cdot n + c_0$ untuk semua $n \geq 0$:

- Basis (n = 0): $T(0) = c_0 = c \cdot 0 + c_0$. Benar.
- Hipotesis: anggap $T(k) = c \cdot k + c_0$ benar untuk suatu $k \geq 0$.
- Langkah induksi: $T(k + 1) = T(k) + c = c \cdot k + c_0 + c = c \cdot (k + 1) + c_0$. Benar.

Hasil Analisis Big-O: untuk n lagu terjadi n + 1 pemanggilan, waktu eksekusi bertambah sebanding dengan n, sehingga growth rate-nya linear dan kompleksitas waktunya O(n).

- **Analisis cariDurasiTerpanjang()**

Fungsi cariDurasiTerpanjang(Lagu[] list, int index) digunakan untuk mencari durasi lagu paling panjang dalam playlist menggunakan rekursi. Fungsi menerima array lagu dan indeks lagu yang sedang diperiksa. Fungsi membandingkan durasi lagu saat ini dengan durasi maksimum dari elemen sebelumnya, kemudian mengembalikan nilai yang lebih besar.

- Base case: Ketika index == 0, fungsi mengembalikan durasi lagu pertama melalui list[0].getDurasi(). Kondisi ini menghentikan pemanggilan rekursif karena tidak ada elemen sebelumnya yang perlu diperiksa.

- Recursive case: Ketika index > 0, fungsi memanggil dirinya sendiri menggunakan parameter index - 1. Fungsi kemudian membandingkan durasi lagu pada indeks saat ini dengan durasi maksimum dari elemen sebelumnya. Jika durasi saat ini lebih besar, fungsi mengembalikan durasi saat ini. Jika tidak, fungsi mengembalikan durasi maksimum sebelumnya.

Karena indeks berkurang satu pada setiap pemanggilan, fungsi akan mencapai base case setelah memeriksa seluruh elemen yang relevan.

Jejak pemanggilan rekursif

Berdasarkan data lima lagu pada program kelompok, proses pencarian dimulai dari indeks terakhir, yaitu indeks 4.

Pemanggilan	Indeks	Durasi saat ini	Maksimum sebelumnya	Hasil
1	4	4.52	4.38	4.52
2	3	4.38	4.15	4.38
3	2	4.15	3.80	4.15
4	1	3.80	3.50	3.80
5	0	3.50	Tidak ada	3.50

tabel di atas memperlihatkan urutan perbandingan saat rekursi kembali dari base case. Nilai maksimum pada setiap tahap diteruskan ke pemanggilan sebelumnya hingga menghasilkan durasi maksimum keseluruhan.

Hasil akhirnya adalah 4.52 menit, yang merupakan durasi lagu Viva La Vida oleh Coldplay pada data playlist kelompok.

Analisis kompleksitas waktu (Big-O)

Misalkan n merupakan jumlah lagu dalam playlist. Setiap pemanggilan rekursif melakukan pekerjaan konstan, yaitu memeriksa indeks, mengambil durasi lagu, dan membandingkan dua nilai. Fungsi kemudian memanggil dirinya sendiri satu kali dengan indeks yang berkurang satu.

Persamaan rekurensnya adalah:

$$T(n) = T(n-1) + c$$

dengan c sebagai konstanta waktu untuk setiap pemanggilan.

Melalui substitusi berulang:

$$T(n) = T(n-1) + c$$

Karena jumlah operasi bertambah sebanding dengan jumlah lagu, kompleksitas waktu fungsi adalah

$O(n)$ Growth rate-nya bersifat linear. Kompleksitas ruangnya juga $O(n)$ karena setiap pemanggilan rekursif menggunakan call stack.

Pengujian hasil pencarian

Pengujian dilakukan dengan membandingkan output fungsi terhadap hasil yang diharapkan berdasarkan data playlist. Tabel berikut merupakan rancangan pengujian yang dapat digunakan setelah program dijalankan.

Jumlah lagu	Durasi maksimum yang diharapkan	Hasil yang diharapkan
0	Tidak ada	Playlist kosong ditangani tanpa memanggil fungsi rekursif
1	3.50 menit	Perfect
3	4.15 menit	Fix You
5	4.52 menit	Viva La Vida
10	Sesuai data 10 lagu	Lagu dengan durasi maksimum pada data pengujian

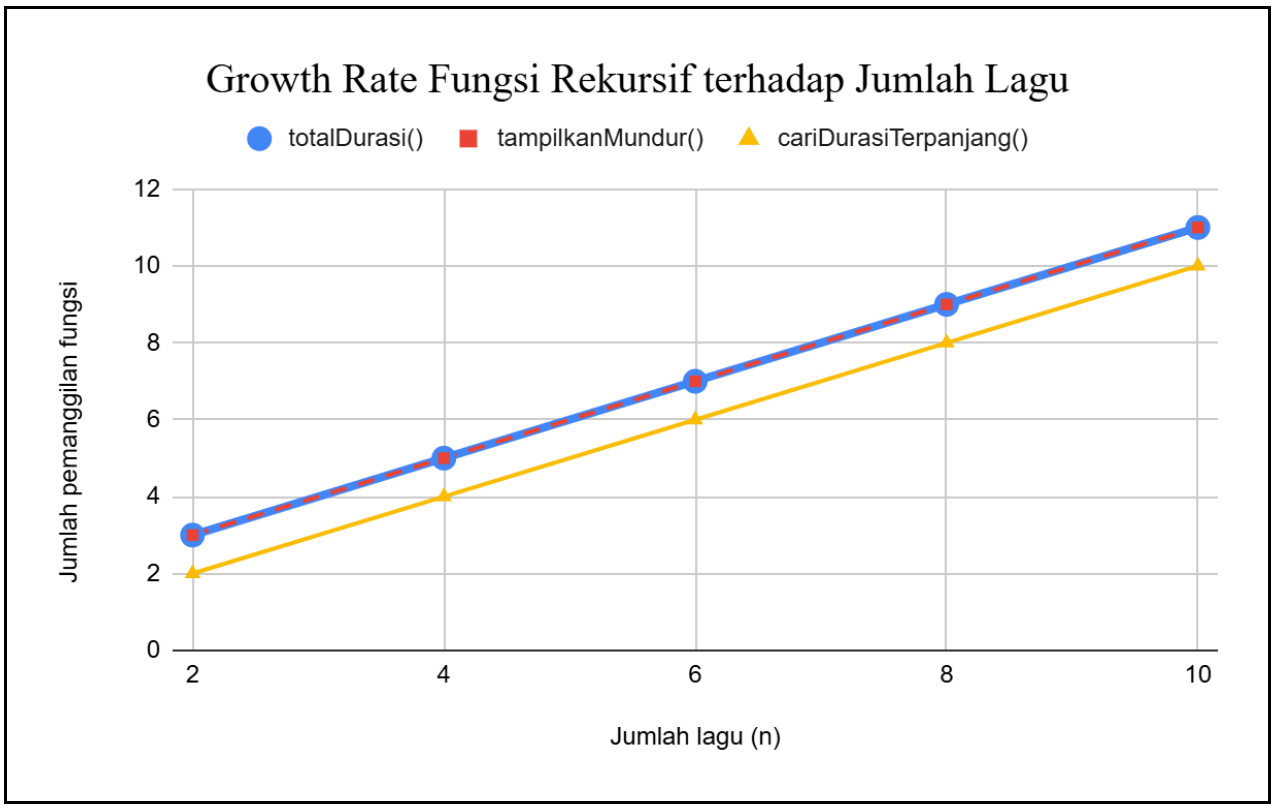
Untuk pengujian 10 lagu, tambahkan lima data uji yang tercantum pada laporan kelompok, yaitu Photograph (4.18), Bad Habits (3.51), The Scientist (5.09), Thinking Out Loud (4.41), dan Paradise (4.38). Berdasarkan data tersebut, durasi maksimum yang diharapkan adalah 5.09 menit, yaitu lagu The Scientist.

Hasil pengujian dinyatakan sesuai apabila durasi yang dikembalikan fungsi sama dengan durasi maksimum hasil pemeriksaan manual. Pengujian aktual dan screenshot perlu diambil setelah program dijalankan.

- **Tabel analisis Big-O dan growth rate**

Fungsi	Base Case	Growth Rate	Kompleksitas
totalDurasi()	n == 0 → mengembalikan 0	Linear	O(n)
tampilkanMundur()	index < 0 → berhenti (return)	Linear	O(n)
cariDurasiTerpanjang()	index == 0 → mengembalikan durasi lagu pertama	Linear	O(n)

- **Grafik growth rate**



Garis totalDurasi() dan tampilkanMundur() bertumpuk karena keduanya dipanggil n + 1 kali. Ketiga garis naik lurus seiring bertambahnya jumlah lagu, sehingga growth rate ketiga fungsi adalah linear (O(n)).

3. Eksperimen Pengukuran Waktu Eksekusi

- **Metode Pengujian**

Pengujian dilakukan untuk mengukur waktu eksekusi tiga fungsi rekursif, yaitu totalDurasi(), tampilkanMundur(), dan cariDurasiTerpanjang(). Pengujian menggunakan tiga variasi jumlah lagu, yaitu 3, 5, dan 10 lagu, untuk mengamati waktu yang dibutuhkan setiap fungsi ketika memproses jumlah data yang berbeda.

Pengukuran dilakukan menggunakan method System.nanoTime() untuk mencatat waktu sebelum dan sesudah fungsi dijalankan. Selisih waktu pengukuran kemudian dikonversi menjadi milidetik (ms). Program dijalankan melalui Eclipse IDE, dan hasil pengujian dicatat berdasarkan output yang ditampilkan pada console.

- **Hasil Pengukuran Waktu Eksekusi**

Hasil pengukuran waktu eksekusi untuk setiap fungsi pada ketiga variasi jumlah lagu disajikan dalam tabel berikut.

Jumlah Lagu (n)	totalDurasi()	tampilkanMundur()	cariDurasiTerpanjang()
3	0 ms	2 ms	0 ms
5	0 ms	1 ms	0 ms
10	0 ms	1 ms	0 ms

- **Dokumentasi Hasil Eksekusi**

Dokumentasi hasil pengujian berupa tangkapan layar console Eclipse IDE disertakan sebagai bukti bahwa program telah dijalankan dengan jumlah data yang berbeda. Pengujian pertama menggunakan 3 lagu, pengujian kedua menggunakan 5 lagu, dan pengujian ketiga menggunakan 10 lagu.

Ketiga tangkapan layar tersebut memperlihatkan jumlah lagu yang diproses, total durasi playlist, lagu dengan durasi terpanjang, urutan lagu yang ditampilkan secara terbalik, serta waktu eksekusi masing-masing fungsi.

```
=== ANALISIS REKURSIF PLAYLIST ===
Jumlah lagu : 3
Total durasi : 11,45 menit
Lagu terpanjang: Fix You
Artis: Coldplay
Durasi: 4,15 menit

Daftar lagu (ditampilkan terbalik):
3. Fix You
2. Shivers
1. Perfect

Execution Time (totalDurasi): 0 ms
Execution Time (tampilkanMundur): 2 ms
Execution Time (cariDurasiTerpanjang): 0 ms
```

```
=== ANALISIS REKURSIF PLAYLIST ===
Jumlah lagu : 5
Total durasi : 20,35 menit
Lagu terpanjang: Viva La Vida
Artis: Coldplay
Durasi: 4,52 menit

Daftar lagu (ditampilkan terbalik):
5. Viva La Vida
4. Yellow
3. Fix You
2. Shivers
1. Perfect

Execution Time (totalDurasi): 0 ms
Execution Time (tampilkanMundur): 1 ms
Execution Time (cariDurasiTerpanjang): 0 ms
```

```
=== ANALISIS REKURSIF PLAYLIST ===
Jumlah lagu : 10
Total durasi : 41,92 menit
Lagu terpanjang: The Scientist
Artis: Coldplay
Durasi: 5,09 menit

Daftar lagu (ditampilkan terbalik):
10. Paradise
9. Thinking Out Loud
8. The Scientist
7. Bad Habits
6. Photograph
5. Viva La Vida
4. Yellow
3. Fix You
2. Shivers
1. Perfect

Execution Time (totalDurasi): 0 ms
Execution Time (tampilkanMundur): 1 ms
Execution Time (cariDurasiTerpanjang): 0 ms
```

- **Analisis Hasil Pengujian**

  Berdasarkan hasil pengujian, ketiga fungsi rekursif berhasil menjalankan tugasnya pada seluruh variasi jumlah lagu. Fungsi totalDurasi() menghitung total durasi lagu dalam playlist, tampilkanMundur() menampilkan daftar lagu dari urutan terakhir ke pertama, sedangkan cariDurasiTerpanjang() menentukan durasi terbesar dari lagu-lagu yang tersedia.

  Hasil pengukuran menunjukkan bahwa waktu eksekusi beberapa fungsi tercatat sebesar 0 ms. Hal ini terjadi karena durasi eksekusi lebih kecil daripada satu milidetik penuh sehingga hasil konversi waktu menjadi milidetik menghasilkan nilai 0. Sementara itu, waktu eksekusi tampilkanMundur() juga mencakup waktu yang digunakan untuk menampilkan teks pada console.

  Dengan demikian, pengujian membuktikan bahwa ketiga fungsi dapat dijalankan pada jumlah data yang berbeda. Namun, hasil pengukuran satu kali eksekusi ini belum cukup untuk menunjukkan perubahan waktu secara konsisten ketika jumlah lagu bertambah. Analisis kompleksitas waktu tetap

didasarkan pada struktur algoritma masing-masing fungsi.

LINK GITHUB: https://github.com/manmaninblack/TugasKelompok3_Group4